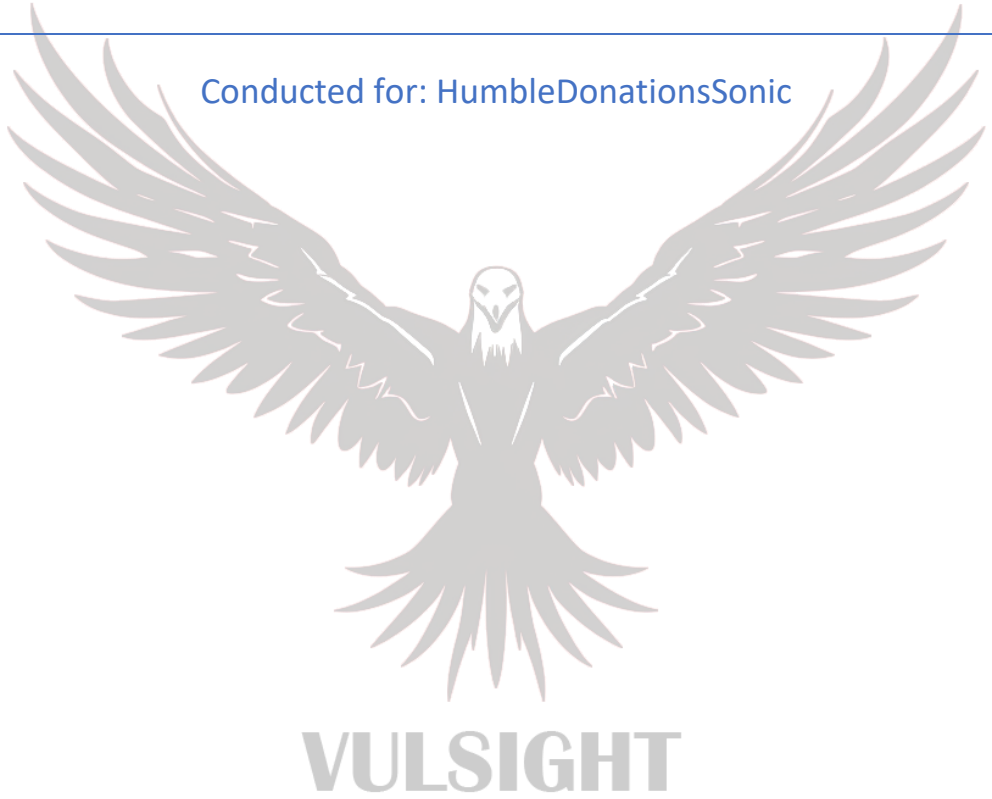
SMART CONTRACT AUDIT

Conducted for: HumbleDonationsSonic



Waleed Ahmed
VULSIGHT
20/12/2024

Contents

Executive Summary:

The client contacted our Consulting Firm to conduct a smart contract audit on their Solidity based smart contract. An audit previously was conducted on the first iteration of the same contract by our firm. This is the second iteration of the same contract.

The Humble Donation smart contract, designed to facilitate charitable donations on EVM compatible block chain, demonstrates a robust architecture for creating and managing donation projects. The contract supports ETH donations as well as multi-token contributions.

Our comprehensive audit revealed no critical or high vulnerabilities. The contract was found to be secure.

Audit conducted by: Waleed Ahmed

Project Background:

The smart contracts are written in solidity. The contract is supposed to be deployed on the Arbitrum Mainnet. The donation contract as well as the token contract are currently deployed on the Sonic Test Net.

Audit Scope:

Deployed Addresses	**XX**
Chain	**Sonic Test Net**
Language	**Solidity**
Audit Completion Date	**20/8/24**

Contract Summary:

- **Contract architecture:** The HumbleDonationsSonic contract utilizes the UUPS (Universal Upgradeable Proxy Standard) pattern, allowing for future upgrades while maintaining data persistence. Use of storage gaps has been made.
- **Token standard implementation:** The contract correctly implements the ERC721 standard for non-fungible tokens, representing individual donation projects.

- **Multi-token support:** The contract efficiently handles donations in various ERC20 tokens as well as native ETH. Instead of Uniswap, the contract integrates with the Solidly swap router) to facilitate token conversions.
- **Access control:** The contract employs OpenZeppelin's Ownable contract to restrict access to critical functions, such as setting tax percentages and withdrawing excess funds, to the contract owner only.
- **Reentrancy protection:** The contract utilizes OpenZeppelin's ReentrancyGuard to protect against reentrancy attacks in critical functions like donate and safeMint.
- **Whitelist mechanism:** The contract implements a Merkle tree-based whitelist for controlling which ERC20 tokens can be used for donations. This provides a gas-efficient way to manage a large list of allowed tokens without storing them all on-chain.
- **Donation and Tax Mechanism:** Donations undergo a tax mechanism that distributes proceeds among designated recipients. The contract does not hold user funds long-term; donations (minus tax) are promptly forwarded to the project's owner.
- **Clear and auditable code:** The contract's code is well organized, commented, and adheres to Solidity best practices. Logical separation of functions and comprehensive documentation promotes auditability and ease of future upgrades.

Various tools such as Slither and different LLM scanners were used in the audit. Extensive manual code review was also done so that any vulnerabilities if present could be detected in the audit.

We found:

- **0 Critical risk finding**
- **0 High risk finding**
- **0 Medium risk findings**
- **0 Low risk finding**

Severity Definitions:

Risk Level	Description
Critical	**Any kind of vulnerability that could lead to direct token or monetary loss**
High	**Any type of vulnerability that could disrupt the proper functioning of smart contract or indirectly lead to token or monetary loss.**
Medium	**Any type of vulnerability that could cause undesired actions but no serious disruption or monetary loss**
Low	**Any type of vulnerability that doesn't have any significant impact on the execution of the proper functioning of contract**

Technical Checks:

Checks	Result
Solidity version not specified	Passed
Solidity version too old	Passed
Integer overflow/underflow	Passed
Function input parameters check bypass	Passed
Insufficient randomness used	Passed
Fallback function misuse	Passed
Race Condition	Passed
Matching Project Specifications	Passed
Logical Vulnerabilities	Passed
Function visibility not explicitly declared	Passed
Use of deprecated keywords/functions	Passed
Out of Gas issue	Passed
Front Running	Passed
Insufficient Decentralization	Passed
Function input parameters lack of check	Passed
Proper function Access Control	Passed
Hardcoded Data	Passed

Code Quality and Documentation:

The audit scope included one smart contract, which was written in Solidity programming language. The code is well indented and clear in nature.

Audit Findings:

Critical findings:

Zero critical findings were found in the smart contract.

High Findings:

Zero high finding were found in the smart contract.

Medium Findings:

Zero medium findings were found in the smart contract.

Low findings:

Zero low findings were found in the smart contract.

Disclaimer:

The smart contract was tested on a best-effort basis. The smart contracts was analyzed with the best security practices known at the time of writing this report. Due to the fact that the total number of test cases is unlimited and the fact that new vulnerabilities in technologies are discovered every day, the audit report makes no guarantees on the security of the contract. While we have done our best in testing this smart contract, it is recommended to not just rely on this audit report alone. A bug bounty program for this smart contract may be created to identify any vulnerabilities within the future.